



TÉCNICO
LISBOA

LINGUAGENS DE PROGRAMAÇÃO

Relatório da 2ª fase do projeto [PT]

Autores:

Daniel Ferreira (112291)

daniel.s.ferreira@tecnico.ulisboa.pt

2024/2025 – 2º Semestre, P4

Conteúdo

1	Introdução	2
2	Verificador de Tipos Estático	2
2.1	Execução base	2
2.2	Implementação de Tipos de Listas	2
2.3	Implementação de estruturas <i>if</i> e <i>match</i>	3
3	Tipos Recursivos e Subtipos	3

1 Introdução

Nesta última fase do projeto de Linguagens de Programação o interpretador da linguagem X++ desenvolvido na primeira fase foi melhorado com um verificador de tipos estático que suporta subtipos e algumas funcionalidades adicionais. Dentro dessas funcionalidades adicionais encontram-se a possibilidade de definir novos tipos, o suporte de tipos produto e soma etiquetados, e o uso do operador "+" para concatenar *strings*. Além disso, é também possível etiquetar a declaração de variáveis e os argumentos nas definições de funções de forma a fazer verificação de tipos sobre o código escrito pelo utilizador.

Durante as restantes secções apresentarei como é que o verificador de tipos estáticos e a implementação de tipos recursivos foram implementados no projeto entregue.

2 Verificador de Tipos Estático

Durante esta secção irei apresentar todos os aspetos importantes da implementação do verificador de tipos estático, passando pela execução geral e por alguns pormenores que foram tidos em conta para a verificação de tipos em estruturas *if*, *match* e de listas, visto irem além do que foi lecionado nas aulas teóricas.

2.1 Execução base

O verificador de tipos estáticos foi elaborado como uma nova passagem pela árvore que é gerada pelo *parser* e que é também usada para avaliar a execução do programa. Agora, em vez de lidar com erros de tipos em tempo de execução, de forma dinâmica, é feita uma passagem prévia pela mesma árvore de forma a garantir que o programa se encontra bem tipificado antes de ser executado. Se esta verificação for efetuada corretamente não poderá existir nenhum erro de tipos durante a execução do programa.

Para implementar este verificador, foi criada uma nova função a ser implementada em todos os tipos de nós *ASTNode* chamada *typecheck*. Esta função recebe dois ambientes: o primeiro relaciona nomes de variáveis a tipos (sendo cada tipo representado por uma classe do tipo *ASTType*) e o segundo relaciona nomes de tipos criados pelo utilizador (com o uso da construção "*type id = tipo;*") a tipos, ou seja, novamente a um objeto da classe *ASTType*. Esta função retorna um objeto da classe *ASTType* e comporta-se de uma forma semelhante à função que avalia o código a ser executado. A função *typecheck* é invocada na cabeça da árvore lida e percorre-a verificando todas as expressões contidas na árvore.

2.2 Implementação de Tipos de Listas

De forma a permitir o maior número de programas seguros possíveis, em vez de considerar que todos os elementos de uma lista devem ser de um mesmo tipo *A*, o meu interpretador permite a construção de listas cujos elementos partilhem um mesmo super-tipo, sendo o tipo inferido pelo interpretador o tipo mais específico que seja super-tipo de todos os elementos contidos na lista. Com isto é por exemplo possível definir a lista " $\{x = 1, y = 2\} :: \{x = 4\} :: nil$ " cujo tipo inferido seria *struct*{*x* : *int*}.

Para implementar a verificação de listas desta forma a lista vai sendo visitada de forma recursiva e é retornado sempre o super-tipo da cauda e da cabeça da lista atual. Se tal não for possível (por exemplo numa lista que contenha um valor inteiro e um booleano) um erro apropriado é gerado. Além disso, foi introduzido um tipo chamado *ASTTAny* que representa qualquer tipo e que é considerado um subtipo universal (que é somente aplicado ao valor *nil* em listas). Desta forma o tipo de uma lista é sempre pelo menos o tipo de um dos valores da lista, pois qualquer tipo é super-tipo de *ASTTAny*.

2.3 Implementação de estruturas *if* e *match*

Na mesma linha de raciocínio, foi também tida em conta a flexibilidade das estruturas de controlo *if* e *match*. Esta implementação do interpretador considera que o tipo de uma destas estruturas corresponde ao super-tipo mais específico dos tipos de cada uma das ramificações. Ou seja, o tipo inferido da estrutura "*if a { {x = 1, y = 2} } else { {x = 1} }*" seria *struct{x : int}*. A implementação deste pormenor é semelhante ao que foi explorado no caso das listas.

Além disto, na estrutura *match* foi também tido em conta que o tipo união aplicado nesta instrução poderá ter somente um subconjunto dos casos da estrutura *match*. Nessas situações, os casos que nunca serão executados por não se encontrarem no tipo usado no argumento da instrução são desconsiderados pois esse código nunca será atingido.

3 Tipos Recursivos e Subtipos

Para que o interpretador conseguisse suportar subtipos foi necessário ter um cuidado especial quando o suporte de subtipos foi implementado. Para tal, para determinar se um tipo *A* é um subtipo de *B*, não basta aplicar a operação de *unfold* até se atingirem tipos primitivos de *A* e *B* que sejam comparáveis. Isto deve-se ao facto de esta operação poder ser executada infinitamente se estes tipos forem realmente recursivos.

Para resolver este problema foi aplicado o algoritmo lecionado nas aulas teóricas. Para a execução deste algoritmo é mantido um conjunto de pares de tipos que já foram visitados durante cada iteração. Em cada iteração verifica-se primeiro se o par de tipos *A* e *B* já foi processado. Se sim, *A* é um subtipo de *B*. Se não, a cada iteração posterior é aplicada a regra de *unfold* até ambos os tipos poderem ser comparados com as restantes regras de *subtyping*.

Por fim, denoto que durante a tipificação de um programa, quando é necessário verificar se um certo identificador tem um tipo específico (por exemplo verificar que os termos de uma multiplicação são valores inteiros), é sempre aplicada a operação de *unfolding* até se atingir um tipo que não seja um identificador, de forma a poder verificar a corretude da expressão.